

VHS Spandau

PR-Maschine

Deployment Guide

Schritt-für-Schritt-Anleitung für Updates, GitHub & Railway

Version 2.0 · Juni 2026

Volkshochschule
Spandau

Übersicht: Der Deployment-Workflow

Schritt	Was passiert	Wer/Was
1	Feature/Fix in Claude besprechen	Du + Claude
2	ZIP herunterladen	Du
3	deploy.bat ausführen (PowerShell)	Du
4	Code landet auf GitHub	Automatisch
5	Railway erkennt neuen Commit	Automatisch
6	Build + Deploy (~60 Sek.)	Railway
7	App ist live	Fertig ✓

Kein manuelles Eingreifen auf dem Server nötig.

Teil 1 — Einmalige Einrichtung

1.1 Voraussetzungen installieren

Tool	Download	Prüfen in PowerShell
Git	https://git-scm.com	git --version
Node.js	https://nodejs.org	node --version

PowerShell öffnen: Windows-Taste → powershell → Enter

```
git --version
node --version
```

1.2 Arbeitsordner anlegen

```
mkdir C:\Users\thoma\Documents\vhsdeploy
cd C:\Users\thoma\Documents\vhsdeploy
```

1.3 GitHub Repository klonen

```
git clone https://github.com/thomas-programmierer/pr-machine.git .
```

Der Punkt am Ende ist wichtig — klonst direkt in den aktuellen Ordner.

1.4 deploy.bat in den Ordner kopieren

Die von Claude gelieferte deploy.bat in C:\Users\thoma\Documents\vhsdeploy\ legen.

1.5 GitHub Zugangsdaten speichern (einmalig)

```
git config --global user.name "Thomas Mochalski"
git config --global user.email "deine@email.de"
git config --global credential.helper manager
```

Beim ersten Push öffnet sich ein Browser-Fenster zur GitHub-Anmeldung — danach werden die Zugangsdaten automatisch gespeichert.

Teil 2 — Standard-Workflow: Update einspielen

Schritt 1 — Neue Version von Claude erhalten

In Claude.ai die gewünschten Änderungen besprechen. Claude liefert eine ZIP-Datei. Diese speichern unter:

```
C:\Users\thoma\Documents\vhsdeploy\downloads\
```

Schritt 2 — PowerShell öffnen

Windows-Taste → powershell → Enter

```
cd C:\Users\thoma\Documents\vhsdeploy
```

Schritt 3 — deploy.bat ausführen

```
.\deploy.bat
```

Das Skript fragt nach dem ZIP-Pfad. Eingeben oder reinziehen:

```
C:\Users\thoma\Documents\vhsdeploy\downloads\vhs-update.zip
```

Schritt 4 — Commit-Nachricht eingeben

Das Skript zeigt welche Dateien geändert wurden und fragt nach einer Beschreibung.

```
Beispiel: Admin-Upload für Kursprogramm und Redaktionsplan
```

Oder einfach Enter drücken für automatisches Datum.

Schritt 5 — Erfolgsmeldung

```
===== Erfolgreich deployt! Railway startet  
automatisch neu. =====
```

Diese Meldung = Code ist auf GitHub, Railway deployt automatisch.

Schritt 6 — Railway abwarten (~60 Sekunden)

Railway Dashboard: <https://railway.app/dashboard>

→ Projekt anklicken → Service → Tab Deployments → Grüner Haken = App ist live

Schritt 7 — App im Browser prüfen

```
https://[app-name].up.railway.app
```

Browser	Cache leeren
Opera	Strg + Shift + R
Chrome	Strg + Shift + R
Firefox	Strg + Shift + R
Edge	Strg + Shift + R

Teil 3 — Railway Dashboard

Deployments-Ansicht

railway.app/dashboard → [Projekt] → [Service] → Deployments

Symbol	Bedeutung
■ Spinner	Build läuft (~60 Sek.)
■ Grüner Haken	App ist live
■ Rotes X	Fehler — Logs ansehen

Manuell neu deployen (falls nötig)

Deployments → letzter Eintrag → Button Redeploy

Bei Fehler: Logs lesen

railway.app/dashboard → [Projekt] → [Service] → Logs

Screenshot machen und Claude schicken — wird sofort analysiert.

Teil 4 — Umgebungsvariablen (Secrets)

Niemals API-Keys in den Code schreiben. Alle Secrets werden in Railway gesetzt:

railway.app/dashboard → [Projekt] → [Service] → Variables

Variable	Wert	Beschreibung
ANTHROPIC_API_KEY	sk-ant-...	Anthropic API für KI-Posts
PORT	(automatisch)	Wird von Railway gesetzt

Teil 5 — GitHub Repository

<https://github.com/thomas-programmierer/pr-machine>

Wichtige Dateien im Repo

Datei	Beschreibung
server.js	Node.js Server — Hauptlogik
public/index.html	Haupt-App (alle Screens)
public/admin.html	Admin-Bereich
railway.json	Railway-Konfiguration
package.json	Node.js Abhängigkeiten
users.json	Benutzerliste
deploy.bat	Deploy-Skript Windows
deploy.sh	Deploy-Skript Mac/Linux
.gitignore	Ausgeschlossene Dateien

Teil 6 — Externe Partner / Dritte

Collaborator einladen

github.com/thomas-programmierer/pr-machine → Settings → Collaborators → Add people

Pull Request Workflow für Dritte

#	Schritt
1	Partner forkt das Repository
2	Macht Änderungen im eigenen Fork
3	Erstellt Pull Request
4	Admin (Thomas) prüft und merged
5	Railway deployt automatisch

API-Endpunkte für externe Integrationen

Endpunkt	Methode	Beschreibung
/api/admin/redaktionsplan-urls	GET	Hinterlegte URLs abrufen
/api/admin/upload	POST	Dateien hochladen (nur Admin)

/api/admin/status	GET	Status Kursprogramm & Redaktionsplan
/api/kurse	GET	Kursliste abrufen
/api/posts	GET	Posts abrufen

Alle Endpunkte erfordern einen eingeloggten Admin-Session-Cookie.

Teil 7 — Zugangsdaten

System	URL / Zugang
App-Login (Admin)	https://[app-name].up.railway.app Benutzer: thomas · Passwort: vhs2026!
GitHub	https://github.com/thomas-programmierer/pr-machine
Railway Dashboard	https://railway.app/dashboard
Claude (Entwicklung)	https://claude.ai

Teil 8 — Häufige Probleme & Lösungen

Problem / Fehlermeldung	Lösung
deploy.bat findet ZIP nicht [FEHLER] Datei nicht gefunden	Pfad prüfen, keine Leerzeichen im Pfad, ZIP wirklich vorhanden?
Push schlägt fehl [FEHLER] Push fehlgeschlagen	Internetverbindung prüfen. Dann: git pull origin main → nochmal .\deploy.bat
Railway zeigt alten Stand	Browser-Cache leeren: Strg + Shift + R Railway → Deployments → Redeploy
App startet nicht (Railway Fehler)	Railway → Logs ansehen. Häufigste Ursache: ANTHROPIC_API_KEY fehlt in Variables
Neue Version nicht sichtbar	60 Sekunden warten → Strg + Shift + R → Railway prüfen ob Build grün ist

Schnellreferenz — Die 4 PowerShell-Befehle

#	Befehl	Wann
1	cd C:\Users\thoma\Documents\vhsdeploy	Immer zuerst
2	.\deploy.bat	Jedes Update
3	git pull origin main	Bei Konflikten
4	railway.app → Redeploy	Notfall-Deploy